

1. Programación en la Base de Datos

Hasta ahora, hemos visto SQL como un lenguaje para realizar consultas y modificaciones simples. Sin embargo, bases de datos como MySQL permiten almacenar lógica de programación completa dentro del propio servidor. Esto se logra mediante bloques de código que encapsulan procesos complejos, mejorando el rendimiento (al reducir el tráfico de red) y la seguridad.

Concepto Clave: El DELIMITER

En MySQL, el símbolo `;` se usa para terminar una instrucción. Al escribir procedimientos o funciones que contienen múltiples líneas con sus propios `;`, MySQL se confundiría y terminaría la ejecución prematuramente. Para solucionarlo, usamos `DELIMITER //` para cambiar temporalmente el símbolo de cierre, escribimos nuestro bloque, y luego lo devolvemos con `DELIMITER ;`.

2. Procedimientos Almacenados (Stored Procedures)

Un Procedimiento Almacenado es un bloque de código SQL preparado que puedes guardar y reutilizar. Puede contener múltiples sentencias (`SELECT`, `INSERT`, `UPDATE`) y lógica condicional (`IF`, `WHILE`).

Creación de un Procedimiento Básico

Imagina que todos los días necesitas ejecutar un reporte complejo. En lugar de escribir la consulta cada vez, la guardas en un procedimiento.

```
-- Cambiamos el delimitador temporalmente DELIMITER // CREATE PROCEDURE
ObtenerClientesRecientes() BEGIN SELECT nombre, email, fecha_registro FROM clientes
WHERE fecha_registro >= DATE_SUB(CURDATE(), INTERVAL 30 DAY) ORDER BY
fecha_registro DESC; END // -- Restauramos el delimitador normal DELIMITER ;
```

Llamando a un Procedimiento

Para ejecutar el procedimiento guardado, utilizamos el comando `CALL`.

```
CALL ObtenerClientesRecientes();
```

Procedimientos con Parámetros

Los procedimientos se vuelven realmente útiles cuando les pasamos variables (parámetros). Existen tres tipos de parámetros en MySQL:

- **IN:** (Por defecto) Se envía información *hacia* el procedimiento.
- **OUT:** El procedimiento devuelve un valor a través de esta variable.
- **INOUT:** La variable entra con un valor, se modifica dentro, y sale con el nuevo valor.

```
-- Procedimiento que recibe un ID y devuelve el saldo total en una variable OUT
DELIMITER // CREATE PROCEDURE ConsultarSaldo(IN p_cliente_id INT, OUT p_saldo
DECIMAL(10,2)) BEGIN SELECT SUM(monto) INTO p_saldo FROM pagos WHERE cliente_id =
p_cliente_id; END // DELIMITER ; -- Ejecutando y mostrando el resultado usando una
variable de sesión (@variable) CALL ConsultarSaldo(5, @saldo_actual); SELECT
@saldo_actual AS Saldo_De_Cliente;
```

3. Funciones Definidas por el Usuario (UDF)

A diferencia de los procedimientos (que ejecutan acciones complejas y se llaman con CALL), una **Función** está diseñada estrictamente para realizar un cálculo, retornar un único valor, y usarse *dentro* de otras sentencias SQL (como en un SELECT o WHERE).

Características de las Funciones

- Siempre deben retornar un valor mediante la instrucción RETURN.
- No pueden modificar la base de datos (no pueden ejecutar INSERT, UPDATE, DELETE).
- Se utilizan como si fueran funciones nativas de MySQL (como SUM() o COUNT()).

DETERMINISTIC vs NOT DETERMINISTIC

Al crear una función, MySQL requiere que indiques si es DETERMINISTIC (si le pasas los mismos parámetros, siempre devolverá exactamente el mismo resultado) o NOT DETERMINISTIC (el resultado puede variar, por ejemplo, si depende de la hora actual).

Ejemplo: Creación de una Función

Vamos a crear una función que calcule un impuesto (IVA del 19%) basándose en el precio base de un producto.

```
DELIMITER // CREATE FUNCTION CalcularIVA(precio_base DECIMAL(10,2)) RETURNS  
DECIMAL(10,2) DETERMINISTIC BEGIN DECLARE impuesto DECIMAL(10,2); SET impuesto =  
precio_base * 0.19; RETURN impuesto; END // DELIMITER ;
```

Ahora podemos utilizar esta función directamente en nuestras consultas habituales:

```
-- Usando la función dentro de un SELECT SELECT nombre_producto, precio,  
CalcularIVA(precio) AS valor_iva, (precio + CalcularIVA(precio)) AS precio_total  
FROM productos;
```

Diferencias Clave: Procedimiento vs. Función

Característica	Procedimiento Almacenado	Función (UDF)
Propósito	Ejecutar flujos de trabajo y operaciones CRUD.	Realizar cálculos y devolver un valor simple.
Invocación	Se llama con el comando CALL nombre();	Se usa en sentencias SQL: SELECT nombre();
Retorno de Datos	Puede no devolver nada, devolver múltiples conjuntos de datos o usar parámetros OUT.	Debe devolver un solo valor obligatoriamente usando RETURN.
Modificación (DML)	Permitida. Puede hacer INSERT, UPDATE, DELETE.	Prohibida. Solo operaciones de lectura permitidas.

4. Triggers (Disparadores)

Un Trigger es un programa especial que se ejecuta **automáticamente** en respuesta a un evento específico en una tabla. Piensa en ellos como alarmas: "Si alguien hace X, entonces haz Y de forma silenciosa".

Eventos y Tiempos de Ejecución

- **Eventos:** Un trigger reacciona a los comandos INSERT, UPDATE, o DELETE.
- **Tiempos:** Pueden configurarse para dispararse BEFORE (antes) o AFTER (después) de que el evento modifique la tabla.

Las variables virtuales: NEW y OLD

Dentro de un trigger, MySQL proporciona dos pseudo-registros que te permiten acceder a los datos:

- **NEW:** Representa los datos *nuevos* que están a punto de insertarse o actualizarse. (Solo existe en INSERT/UPDATE).
- **OLD:** Representa los datos *viejos* antes de ser modificados o eliminados. (Solo existe en UPDATE/DELETE).

Ejemplo: Trigger de Auditoría

Supongamos que queremos mantener un historial (log) de cada vez que se actualiza el salario de un empleado. Primero, tenemos una tabla `historial_salarios`. Ahora creamos el trigger en la tabla `empleados`.

```
DELIMITER // CREATE TRIGGER auditar_cambio_salario AFTER UPDATE ON empleados FOR EACH ROW BEGIN -- Verificamos si realmente hubo un cambio en la columna salario IF NEW.salario != OLD.salario THEN INSERT INTO historial_salarios (empleado_id, salario_viejo, salario_nuevo, fecha) VALUES (OLD.id, OLD.salario, NEW.salario, NOW()); END IF; END // DELIMITER ;
```

¿Qué sucede aquí? Cada vez que alguien ejecute un comando `UPDATE empleados SET salario = ...`, este trigger despertará automáticamente. Si el valor del salario cambió, registrará la modificación de forma segura sin requerir que la aplicación principal haga inserciones adicionales manuales.